COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 DSA	**SECTION:**	CPE21S1
DATE PERFORMED:	07/16/26	**DATE SUBMITTED:**	07//26
NAME:	Aguila, Daniel C.	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. ___ [ACTIVITY NAME]

1. PROCEDURE OUTPUT

What procedures did you perform?

SCREENSHOT

Code:

```cpp
#include <iostream>
#include <string.h>

class Student{
    private:
    std::string studentName;
    int studentAge;

    public:
    //constructor
    Student(std::string newName = "John Doe", int newAge = 18){
        studentName = std::move(newName);
        studentAge = newAge;
        std::cout << "Constructor Called." << std::endl;
    };

    //deconstructor
    ~Student(){
        std::cout << "Deconstructor Called." << std::endl;
    }

    //Copy Constructor
    Student(const Student &copyStudent){
        std::cout << "Copy Constructor Called." << std::endl;
        studentName = copyStudent.studentName;
        studentAge = copyStudent.studentAge;
    }
```

```
    }

    //Display Attributes
    void printDetails(){
        std::cout << this->studentName << " " << this->studentAge << std::endl;
    }
};

int main() {
    const size_t j = 5;
    Student studentList[j] = {};
    std::string namesList[j] = {"Carly", "Freddy", "Sam", "Zach", "Cody"};
    int ageList[j] = {15, 16, 18, 19, 16};

    return 0;
}
```

Output:

```
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Deconstructor Called.
Deconstructor Called.
Deconstructor Called.
Deconstructor Called.
Deconstructor Called.
PS C:\Users\TIPQC\Desktop\Aguila CPE21S1 New\hoa_2>
```

OBSERVATION

-The Student class defines a constructor that initializes each object with a default name and age, and prints a message whenever a new object is created. Similarly, the destructor prints a message when an object is destroyed, showing the end of its lifecycle.

-In the main() function, an array of five Student objects is declared. As soon as the array is created, the constructor runs five times once for each element which is why the output shows “Constructor Called” five times. When the program finishes, all objects go out of scope, and the destructor runs five times, printing “Destructor Called” for each one.

Table 2-1. Initial Driver Program

SCREENSHOT

Code:

```cpp
#include <iostream>
#include <string.h>

class Student{
    private:
    std::string studentName;
    int studentAge;

    public:
    //constructor
    Student(std::string newName = "John Doe", int newAge = 18){
        studentName = std::move(newName);
        studentAge = newAge;
        std::cout << "Constructor Called." << std::endl;
    };

    //deconstructor
    ~Student(){
        std::cout << "Deconstructor Called." << std::endl;
    }

    //Copy Constructor
    Student(const Student &copyStudent){
        std::cout << "Copy Constructor Called." << std::endl;
        studentName = copyStudent.studentName;
        studentAge = copyStudent.studentAge;
    }

```

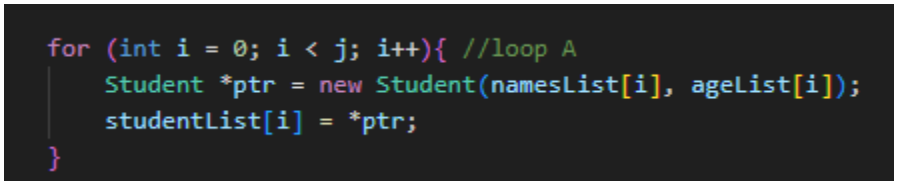
```
29      //Display Attributes
30      void printDetails(){
31          std::cout << this->studentName << " " << this->studentAge << std::endl;
32      }
33  };
34
35  int main() {
36      const size_t j = 5;
37      Student studentList[j] = {};
38      std::string namesList[j] = {"Carly", "Freddy", "Sam", "Zach", "Cody"};
39      int ageList[j] = {15, 16, 18, 19, 16};
40
41      for (int i = 0; i < j; i++){ //loop A
42          Student *ptr = new Student(namesList[i], ageList[i]);
43          studentList[i] = *ptr;
44      }
45
46      for (int i = 0; i < j; i++){ //loop B
47          studentList[i].printDetails();
48      }
49
50      return 0;
51  }
```

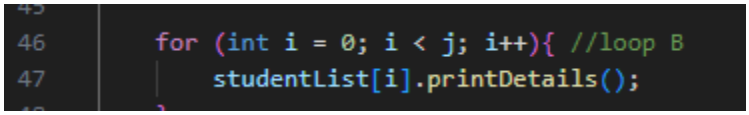
Output:

```
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Carly 15
Freddy 16
Sam 18
Zach 19
Cody 16
Deconstructor Called.
Deconstructor Called.
Deconstructor Called.
Deconstructor Called.
Deconstructor Called.
PS C:\Users\TIPQC\Desktop\Aguila CPE21S1 New\hoa_2>
```

OBSERVATION
-In the first loop, each Student object is dynamically created using the new keyword, which triggers the constructor and initializes the student's name and age. When these objects are copied into the studentList array, the copy constructor is called, showing how data is duplicated between objects -In the second loop, the program prints each student's name and age, confirming that the data was successfully transferred and stored. When the program ends, destructors are called for each object, releasing memory and signaling that the objects are destroyed.

Table 2-2. Modified Driver Program with Student Lists

LOOP A

OBSERVATION
-The loop runs five times, creating one Student object per iteration. -Each object is dynamically allocated using new, which triggers the constructor and assigns the student's name and age. -The pointer ptr temporarily holds the address of the new object. -The statement studentList[i] = *ptr; copies the object's data into the array, activating the copy constructor.

LOOP B

OBSERVATION
-The loop runs five times, matching the number of students created earlier. Each iteration calls the printDetails() function for one Student object in the studentList array.

-The function displays the student's name and age and it confirms that the data was correctly copied and stored. This loop focuses on output and verification that shows the program successfully transferred information from dynamically created objects to the array. Also, loop B highlights the retrieval and display phase of the program

OUTPUT

```
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Carly 15
Freddy 16
Sam 18
Zach 19
Cody 16
Deconstructor Called.
Deconstructor Called.
Deconstructor Called.
Deconstructor Called.
Deconstructor Called.
PS C:\Users\TIPQC\Desktop\Aguila CPE21S1 New\hoa_2>
```

OBSERVATION

- The first ten lines show "Constructor Called", which means the constructor was executed ten times five times for the initial array studentList and five more times for the dynamically created objects inside Loop A.
- Each time a new Student object is created using new, the constructor initializes the student's name and age, confirming successful object creation. After that, the program prints the names and ages Carly 15, Freddy 16, Sam 18, Zach 19, Cody 16 that shows that the data was correctly copied and stored in the studentList array.
- The last five lines display "Deconstructor Called", meaning the destructors were triggered when the program ended and the studentList objects went out of scope.

MODIFICATIONS

Code:

```
int main() {
    const size_t j = 5;
    Student studentList[j] = {};
    std::string namesList[j] = {"Carly", "Freddy", "Sam", "Zach", "Cody"};
    int ageList[j] = {15, 16, 18, 19, 16};

    for (int i = 0; i < j; i++){ //loop A
        Student *ptr = new Student(namesList[i], ageList[i]);
        studentList[i] = *ptr;
        delete ptr;
    }

    for (int i = 0; i < j; i++){ //loop B
        studentList[i].printDetails();
    }

    return 0;
}
```

Output:

```
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Deconstructor Called.
Constructor Called.
Deconstructor Called.
Constructor Called.
Deconstructor Called.
Constructor Called.
Deconstructor Called.
Constructor Called.
Deconstructor Called.
Carly 15
Freddy 16
Sam 18
Zach 19
Cody 16
Deconstructor Called.
Deconstructor Called.
Deconstructor Called.
Deconstructor Called.
Deconstructor Called.
PS C:\Users\TIPQC\Desktop\Aguila CPE21S1 New\hoa_2>
```

OBSERVATION

-In this updated version, the delete ptr; statement was added to properly release the memory allocated by new, preventing leaks. Each iteration of Loop A creates a temporary Student object, copies its data into the array, and then deletes it, causing both the constructor and destructor to run within the same loop. This results in alternating “Constructor Called” and “Deconstructor Called” messages in the output. Loop B then prints each student’s name and age, confirming that the data was successfully stored in the studentList array.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

COMPUTER ENGINEERING

QUEZON CITY

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.